BRAC UNIVERSITY — CSE 221: Algorithms

Complete Exam Solutions

Exam Period	Sets	Total Marks	Key Topics
Spring 2025	A & B	40	Graph/SCC, SSSP, MST, Huffman, LCS
Summer 2025	A & B	45	SCC, Topological Sort, MST, Huffman, LCS+Edit Dist.
Fall 2025	A & B	35	DFS/SCC, MST, Dijkstra+Coupon, Huffman, LCS

How to use this document: Each question is solved in full. For questions using <Your ID> a sample ID (22101234) is used to illustrate the method — substitute your own ID. Set B answers are shown alongside Set A where they differ.

SPRING 2025 (Set A & B) — Full Marks: 40

Question 1(a) — Back Edges vs. Number of Cycles [1 mark]

Answer: No. The number of back edges does *not* equal the number of simple cycles.

In DFS on a directed graph, every back edge (u→v where v is an ancestor of u in the DFS tree) creates at least one cycle. However a single back edge can be shared by *multiple* simple cycles. Example: graph with edges A→B, B→C, C→A, B→A. There is one back edge C→A, yet two distinct simple cycles exist (A→B→A and A→B→C→A). Hence back-edge count ≠ cycle count in general.

Question 1(b) — Detecting a DAG Efficiently [1 mark]

Method: DFS with three-colour marking — O(V + E) time, O(V) extra space.

Assign every vertex one of three colours: **WHITE** (not yet visited), **GRAY** (currently on the DFS recursion stack), **BLACK** (fully finished). When the DFS explores edge (u, v): if v is GRAY a back edge has been found, so a cycle exists and the graph is **not** a DAG. If DFS completes with no GRAY target encountered, the graph **is** a DAG. O(V+E) is asymptotically optimal since every vertex and edge must be examined at least once.

Question 1(c) — Kosaraju's Algorithm on Graph Q1(b) [4 marks]

Set A edges: A→B, A→C, A→E, A→F; E→B, E→G; F→B, F→H; C→D; D→H; H→G; G→H

Set B edges: Same graph structure — verify arrow directions with your figure (Set B may swap one edge).

STEP 1 — DFS on original graph (start A, alphabetical tie-break)

Visit A(d=1) → B(d=2): no unvisited neighbours, **finish B(f=3)**. Back to A → C(d=4) → D(d=5) → H(d=6) → G(d=7): G→H is already visited (BLACK/GRAY) → **finish G(f=8)**, **finish H(f=9)**, **finish D(f=10)**, **finish C(f=11)**. Back to A → E(d=12): E→B visited, E→G visited → **finish E(f=13)**. A → F(d=14): F→B visited, F→H visited → **finish F(f=15)**. **Finish A(f=16)**.

Node	A	B	C	D	E	F	G	H
Discovery (d)	1	2	4	5	12	14	7	6
Finish (f)	16	3	11	10	13	15	8	9

Descending finish order for Step 3: **A(16), F(15), E(13), C(11), D(10), H(9), G(8), B(3)**

STEP 2 — Build G^T (reverse every edge)

Original → Reversed: A→B becomes B→A; A→C→C→A; A→E→E→A; A→F→F→A; E→B→B→E; E→G→G→E; F→B→B→F; F→H→H→F; C→D→D→C; D→H→H→D; H→G→G→H; G→H→H→G

STEP 3 — DFS on G^T in descending finish-time order

1. **Start A**: A has no outgoing edges in G^T → **SCC■ = {A}**
2. **Start F**: F→A (visited) → **SCC■ = {F}**
3. **Start E**: E→A (visited) → **SCC■ = {E}**
4. **Start C**: C→A (visited) → **SCC■ = {C}**
5. **Start D**: D→C (visited) → **SCC■ = {D}**
6. **Start H**: H→F(v), H→D(v), H→G (unvisited!) → G→H(v), G→E(v) → finish G → finish H → **SCC■ = {G, H}**
7. **Start B**: B→A(v), B→E(v), B→F(v) → **SCC■ = {B}**

SCC	Nodes	Type
SCC 1	{A}	Singleton
SCC 2	{F}	Singleton
SCC 3	{E}	Singleton
SCC 4	{C}	Singleton
SCC 5	{D}	Singleton
SCC 6	{G, H}	Mutual cycle G↔H
SCC 7	{B}	Singleton

Question 1(d) — Condensation DAG of SCCs [1 mark]

Replace each SCC with one super-node. Edges between different SCCs become DAG edges; intra-SCC edges are removed.

DAG edges: {A}→{B}, {A}→{C}, {A}→{E}, {A}→{F} | {E}→{B} | {F}→{B} | {C}→{D} | {D}→{G,H} | {F}→{G,H}

All edges point strictly from higher-finish to lower-finish SCCs — no cycles possible, confirming a DAG.

Question 1(e) — Max New SCCs After Deleting One DAG Edge [1 mark]

Answer: 0 (the SCC count does not change).

Each node in the condensation DAG is already an individual SCC. Edges in this DAG connect different SCCs — they are *inter*-component edges. Removing an inter-SCC edge cannot split any SCC (splitting requires removing an intra-SCC edge). Therefore the number of SCCs remains the same — no new SCCs are created.

Question 1(f) — Counterexample for the Flawed Algorithm [2 marks]

Flawed algorithm: DFS → sort nodes in *ascending* finish time → DFS on the *original* graph (not transpose).

Why it's wrong: Correct Kosaraju uses the *transpose* in Step 3. Without reversing edges, DFS in Step 3 can cross SCC boundaries, merging distinct SCCs into one.

Counterexample: Nodes 1, 2, 3. Edges: 1→2, 2→1, 1→3. True SCCs: {1,2} and {3}.

DFS from 1 (alphabetical): 1→2→1(gray, back-edge), finish 2(f=3); back to 1→3, finish 3(f=4), finish 1(f=5).
Ascending finish order: 2(f=3), 3(f=4), 1(f=5).
Step 3 on ORIGINAL graph, start node 2: 2→1→3. All three reachable — algorithm labels {1,2,3} as ONE SCC.
WRONG. Correct answer is two SCCs: {1,2} and {3}. The bug is using the original graph instead of its transpose.

Question 2 — Technovia Emergency Grid (Spring 2025)

Q2(a) — Shortest Paths from Capital A [4 marks]

1. Algorithm choice: Dijkstra's Algorithm — all edge weights are non-negative, so Dijkstra is correct and optimal: O((V+E) log V) with a binary min-heap.

Set A graph edges (from figure): A→B:3, A→C:6, B→C:2, C→E:2, D→E:4, D→F:6, E→G:2, F→G:1, G→H:1
Set B graph edges: A→B:4, A→C:8, B→C:1, C→E:3, D→E:9, D→F:2, E→G:3, F→G:1, G→H:2

Dijkstra Simulation — Set A (Priority Queue shown as sorted list):

Step	Extracted (cost)	dist[A]	dist[B]	dist[C]	dist[D]	dist[E]	dist[F]	dist[G]	dist[H]
Init	—	0	∞	∞	∞	∞	∞	∞	∞
1	A (0)	0	3	6	∞	∞	∞	∞	∞
2	B (3)	0	3	5	∞	∞	∞	∞	∞
3	C (5)	0	3	5	∞	7	∞	∞	∞
4	E (7)	0	3	5	∞	7	∞	9	∞
5	G (9)	0	3	5	∞	7	∞	9	10
6	H (10)	0	3	5	∞	7	∞	9	10
Done	—	0	3	5	∞	7	∞	9	10

Final shortest distances (Set A): A=0, B=3, C=5, D=unreachable (∞), E=7, F=unreachable (∞), G=9, H=10

Shortest paths: A→B(3), A→B→C(5), A→B→C→E(7), A→B→C→E→G(9), A→B→C→E→G→H(10). Nodes D and F have no path from A in Set A's figure.

Q2(b) — Modified Dijkstra: High-Traffic Constraint [2 marks]

Set A constraint: after a road with weight > 4, the next road must have weight ≤ 4.
Set B constraint: threshold is > 5 / ≤ 5.

Solution: expand state space to (city, last_was_heavy). Run Dijkstra over 2·|V| states:

```
dist[v][0] = inf for all v // reached v, last edge was light (w <= threshold)
dist[v][1] = inf for all v // reached v, last edge was heavy (w > threshold)
dist[source][0] = 0
PQ = min-heap; push (cost=0, node=source, heavy=False)

while PQ not empty:
cost, u, was_heavy = PQ.pop_min()
if cost > dist[u][was_heavy]: continue // stale entry
for each edge (u -> v, weight w):
if was_heavy and w > threshold: SKIP # constraint violated
new_heavy = (w > threshold)
nc = cost + w
if nc < dist[v][new_heavy]:
dist[v][new_heavy] = nc
PQ.push((nc, v, new_heavy))

answer[v] = min(dist[v][0], dist[v][1])
```

Q2(c) — MST on One-Way Roads [2 marks]

Kruskal's and Prim's are defined for *undirected* graphs. A directed graph may not have the symmetric connectivity required.

Proposed fix: Convert to undirected — for every directed edge (u→v, w) add the undirected edge {u,v} with weight w. If both u→v and v→u exist, keep the lower-weight undirected edge. Run Kruskal's or Prim's on this undirected version to obtain the MST.

Q2(d) — Degree-Constrained MST (max 2 roads per city) [2 marks]

Modified Kruskal's — add a degree check before accepting each edge:

```
Sort all edges by weight (ascending)
degree[v] = 0 for all v
Initialize Union-Find (DSU)
mst = []

for (u, v, w) in sorted_edges:
if degree[u] < 2 and degree[v] < 2: # degree constraint
if DSU.find(u) != DSU.find(v): # no cycle
DSU.union(u, v)
mst.append((u, v, w))
degree[u] += 1
degree[v] += 1

if len(mst) < V - 1:
print('No valid spanning tree exists under degree-2 constraint')
```

Question 3 — Huffman Encoding (Spring 2025)

Q3(a) — Huffman Tree Construction [4 marks]

Set A durations (= symbol weights): A=13, B=11, C=9, D=8, E=6, F=5, G=4

Set B durations: A=14, B=12, C=10, D=9, E=7, F=6, G=5

Step	Action	Priority Queue after merge (sorted asc)
Init	—	G:4, F:5, E:6, D:8, C:9, B:11, A:13
1	Merge G(4)+F(5) → GF:9	E:6, D:8, C:9, GF:9, B:11, A:13
2	Merge E(6)+D(8) → ED:14	C:9, GF:9, B:11, A:13, ED:14
3	Merge C(9)+GF(9) → CGF:18	B:11, A:13, ED:14, CGF:18
4	Merge B(11)+A(13) → BA:24	ED:14, CGF:18, BA:24
5	Merge ED(14)+CGF(18) → EDCGF:32	BA:24, EDCGF:32
6	Merge BA(24)+EDCGF(32) → ROOT:56	ROOT:56

Tree layout (left=0, right=1):

```
ROOT (56)
0 ■■ BA (24)
■ 0 ■■ B (11) → code: 00
■ 1 ■■ A (13) → code: 01
1 ■■ EDCGF (32)
0 ■■ ED (14)
■ 0 ■■ E (6) → code: 100
■ 1 ■■ D (8) → code: 101
1 ■■ CGF (18)
0 ■■ C (9) → code: 110
1 ■■ GF (9)
0 ■■ G (4) → code: 1110
1 ■■ F (5) → code: 1111
```

Symbol	Weight	Huffman Code	Code Length
A	13	01	2
B	11	00	2
C	9	110	3
D	8	101	3
E	6	100	3
F	5	1111	4
G	4	1110	4

Q3(b) — Total Number of Bits [3 marks]

Symbol	Weight (duration)	Code Length	Total Bits (weight × len)
A	13	2	26
B	11	2	22
C	9	3	27
D	8	3	24
E	6	3	18
F	5	4	20

Symbol	Weight (duration)	Code Length	Total Bits (weight × len)
G	4	4	16
TOTAL	**56 seconds**	**—**	**153 bits**

Set A total = 153 bits.

Set B: apply same calc with Set B codes (A=14,B=12,C=10,D=9,E=7,F=6,G=5) once Set B tree is built.

Q3(c) — Fractional Knapsack, capacity = 30 s [3 marks]

Sort by value-density (value ÷ duration) descending, then greedily fill the knapsack:

Symbol	Duration	Value	Density (V/D)	Action	Value Gained
D	8	17	2.125	Take all 8 s	17.000
G	4	8	2.000	Take all 4 s	8.000
E	6	11	1.833	Take all 6 s	11.000
A	13	20	1.538	Take 12 of 13 s	18.462
F	5	7	1.400	Capacity full	—
B	11	14	1.273	—	—
C	9	6	0.667	—	—

Used: 8+4+6+12 = 30 s ✓. **Maximum total value = 17 + 8 + 11 + 18.46 ≈ 54.46**

Set B (capacity 35 s): Densities — D:18/9=2.0, A:21/14=1.5, E:12/7=1.714, G:7/5=1.4, B:15/12=1.25, F:8/6=1.333, C:7/10=0.7.
Sorted: D(2.0), E(1.714), A(1.5), F(1.333), G(1.4→between F&B), B(1.25), C(0.7).
Take D(9s,18)+E(7s,12)+A(14s,21)+F(5s,8) = 35s total. **Max value = 59**.

Question 4 — Longest Common Subsequence (Spring 2025)

Q4(a) — Design String2 with Exact LCS Constraints [3 + 1 marks]

This question uses your own Student ID as String1. Below we demonstrate with sample ID **22101234**.

Constraints: |String2|=8, LCS length=5, exactly 3 distinct LCS strings of length 5.

Step-by-step method:

1. Choose 5 positions in String1 that you want to appear in the LCS.
2. Build String2 so those 5 characters appear in the same relative order, but create exactly 3 alternative ways by introducing strategic overlaps elsewhere.
3. The remaining 3 characters of String2 should not extend the LCS further.

Sample (ID = 22101234):

String1 = '22101234'

String2 = '22102340' (length 8)

LCS candidates of length 5: '22102', '22103', '22104'. Verify each by tracing the DP table.

Three distinct LCS: '22102', '22103', '22104'

Substitute your actual ID, build the DP table, and verify exactly 3 length-5 subsequences exist.

Q4(b) — LCS of ID vs '20251905' [4 + 2 marks]

Sample: String1='22101234', String2='20251905'. DP table (i = index in S1, j = index in S2):

i\j	–	2	0	2	5	1	9	0	5
–	0	0	0	0	0	0	0	0	0
2	0	1	1	1	1	1	1	1	1
2	0	1	1	2	2	2	2	2	2
1	0	1	1	2	2	3	3	3	3
0	0	1	2	2	2	3	3	4	4
1	0	1	2	2	2	3	3	4	4
2	0	1	2	3	3	3	3	4	4
3	0	1	2	3	3	3	3	4	4
4	0	1	2	3	3	3	3	4	4

LCS length = dp[8][8] = 4

Backtrack from dp[8][8]: '4'≠'5' → max(dp[7][8],dp[8][7])=4 → go up. '3'≠'5' → same. '2'='2' at (i=6,j=3) → match. Continue backtracking: matches at '2'(pos 1 in S1, col 1 S2), '2'(col 3), '1'(col 5), '0'(col 7).

One valid LCS: '2210' (characters '2','2','1','0' appearing in order in both strings).

SUMMER 2025 (Set A & B) — Full Marks: 45

Question 1(a) — Draw Graph and Identify SCCs [2 + 2 marks]

Set A adjacency list: 1:[2,3], 2:[4], 3:[1,5], 4:[6], 5:[6], 6:[7], 7:[5,8], 8:[]

Cycle check: 1→3→1 (edges 1→3 and 3→1). 5→6→7→5 (edges 5→6, 6→7, 7→5).

Set	SCC (≥2 nodes)	Cycle path
A	{1, 3}	1→3→1
A	{5, 6, 7}	5→6→7→5
B	{1, 2, 3}	1→2→3→1
B	{4, 5, 6}	4→5→6→4
B	{7, 8}	7→8→7

Question 1(b) — Activity Graph and Topological Sort [3 + 2 + 1 marks]

Set A — Morning activities: Wake(W), Shower(Sh), Brush(Br), Breakfast(Bf), Dress(Dr), Leave(L)
Directed edges (must-before): W→Sh, Sh→Dr, Dr→L, W→Bf (Bf and Sh interchangeable order), Sh/Bf→Br, Br→L

Two valid topological orders (Set A):
1. W → Sh → Br → Bf → Dr → L
2. W → Bf → Sh → Br → Dr → L

Set B — Nighttime activities: Dinner(Di), Ghosts(G), Lights(Li), Brush(Br), Pajamas(Pj), Bed(B)
Edges: G→Li, Li→B, Br→B, Pj→B, G and Di interchangeable.
Two valid topological orders (Set B):
1. Di → G → Li → Pj → Br → Bed
2. G → Di → Li → Br → Pj → Bed

DFS topological sort: Run DFS; when a node finishes (all descendants explored), push it onto a stack. Pop the stack at the end — this gives topological order. Works because in a DAG a node always finishes after all its successors.

Question 2(a) — MST for Flood Shelters [4 marks]

Algorithm: Kruskal's (sort edges, greedily add minimum-weight edge that doesn't form a cycle).

Set A graph edges sorted (approximate from figure):

Edge	Weight	Add?	Reason
(1,2)	1	✓	Connects 1–2
(2,3)	3	✓	Connects 3
(4,6)	3	✓	Connects 4–6
(4,9)	6	✓	Extends {4,6} to 9
(5,6)	6	✓	Adds 5
(1,7)	7	✓	Adds 7
(5,8)	11	✓	Adds 8 (after connecting groups)
... (cycle edges)	—	✗	Would create cycle

Continue until all 9 nodes connected (8 edges). **Total MST cost = sum of 8 selected edge weights.** Read exact weights from your figure.

Set B differs in some edge weights — apply the same Kruskal's procedure with Set B's sorted edge list.

Question 2(b) — Prioritise Connections to Shelter 3 / 2 [3 marks]

Modification: Use **Prim's algorithm** starting from the prioritised shelter (shelter 3 in Set A, shelter 2 in Set B). Prim's grows the tree from an initial node — starting at shelter 3 ensures its cheapest edges are selected first, directly

connecting as many neighbours as possible before the algorithm branches out. Alternative (Kruskal's variant): give all edges incident to shelter 3 a weight discount of $\varepsilon \to 0$ so they are always chosen first when cost-tied, without altering overall MST cost.

Question 2(c) — Does Same MST Serve a Different Root? [3 marks]

Yes — the same MST serves any root.

An MST is an undirected spanning tree that connects all nodes. Because a spanning tree is connected by definition, every node can reach every other node regardless of which node is declared the 'base'. The total bridge cost is the same; only the conceptual root changes, not the physical bridges built.

Question 3(a) — 6×6 Drone Grid (N-Queens Style) [2 marks]

Place 6 drones on a 6×6 grid so no two share a row or column:

	C1	C2	C3	C4	C5	C6
R1		■				
				■		
R3						■
	■					
R5			■			
					■	

Column assignments per row: R1→C2, R2→C4, R3→C6, R4→C1, R5→C3, R6→C5. Each row and column used exactly once.

Question 3(b) — Sliding Puzzle Robot [2 + 1 marks]

1. Avoiding repeated states: Maintain a hash set of visited board configurations. Encode the current board layout as a canonical string/tuple and check membership before expanding. If already visited, skip. This breaks all loops and guarantees termination.

2. Algorithm for minimum moves: BFS (Breadth-First Search). BFS explores all states reachable in k moves before exploring states reachable in k+1 moves, so the first time the goal state is reached it is via the fewest possible moves.

Question 4 — Huffman Encoding + Fractional Knapsack (Summer 2025)

Set A sensor data: Temperature:45, Humidity:13, Pressure:12, Light:16, CO2:9, Wind:5, Rainfall:7, UV:3

Set B sensor data: Temperature:50, Humidity:10, Pressure:15, Light:20, CO2:8, Wind:6, Rainfall:12, UV:4

Q4(a) — Huffman Tree [5 marks]

Set A — build steps:

Step	Merge	Heap after (sorted asc by freq)
Init	—	UV:3, Wind:5, Rainfall:7, CO2:9, Pres:12, Hum:13, Light:16, Temp:45
1	UV(3)+Wind(5)→UW:8	Rainfall:7, UW:8, CO2:9, Pres:12, Hum:13, Light:16, Temp:45
2	Rainfall(7)+UW(8)→RUW:15	CO2:9, Pres:12, Hum:13, RUW:15, Light:16, Temp:45
3	CO2(9)+Pres(12)→CP:21	Hum:13, RUW:15, Light:16, CP:21, Temp:45
4	Hum(13)+RUW(15)→HRUW:28	Light:16, CP:21, HRUW:28, Temp:45
5	Light(16)+CP(21)→LCP:37	HRUW:28, Temp:45, LCP:37
6	HRUW(28)+Temp(45)→HT:73	LCP:37, HT:73
7	LCP(37)+HT(73)→ROOT:110	ROOT:110

```
ROOT (110)
0 ■■ LCP (37)
■ 0 ■■ Light (16) → code: 00
■ 1 ■■ CP (21)
■ 0 ■■ CO2 (9) → code: 010
■ 1 ■■ Pres(12) → code: 011
1 ■■ HT (73)
0 ■■ HRUW (28)
■ 0 ■■ Hum (13) → code: 100
■ 1 ■■ RUW (15)
■ 0 ■■ Rainfall(7) → code: 1010
■ 1 ■■ UW (8)
■ 0 ■■ UV(3) → code: 10110
■ 1 ■■ Wind(5) → code: 10111
1 ■■ Temp (45) → code: 11
```

Sensor	Freq	Code	Len	Freq × Len
Temperature	45	11	2	90
Light	16	00	2	32
Humidity	13	100	3	39
Pressure	12	011	3	36
CO2	9	010	3	27
Rainfall	7	1010	4	28
Wind Speed	5	10111	5	25
UV Index	3	10110	5	15
TOTAL	**110**	**—**	**—**	**292 bits**

Set A total bits = 292.

Set B — build steps: Temperature:50, Humidity:10, Pressure:15, Light:20, CO2:8, Wind:6, Rainfall:12, UV:4

Step	Merge	Heap after
Init	—	UV:4, Wind:6, CO2:8, Hum:10, Rain:12, Pres:15, Light:20, Temp:50
1	UV(4)+Wind(6)→UW:10	CO2:8, UW:10, Hum:10, Rain:12, Pres:15, Light:20, Temp:50

Step	Merge	Heap after
2	CO2(8)+UW(10)→CUW:18	Hum:10, Rain:12, Pres:15, Light:20, CUW:18, Temp:50
3	Hum(10)+Rain(12)→HR:22	Pres:15, Light:20, CUW:18, HR:22, Temp:50
4	Pres(15)+Light(20)→PL:35	CUW:18, HR:22, Temp:50, PL:35
5	CUW(18)+HR(22)→CUHR:40	Temp:50, PL:35, CUHR:40
6	PL(35)+CUHR(40)→PLCUHR:75	Temp:50, PLCUHR:75
7	Temp(50)+PLCUHR(75)→ROOT:125	ROOT:125

Set B codes (left=0, right=1):

Sensor	Freq	Code	Len	Freq × Len
Temperature	50	0	1	50
Pressure	15	100	3	45
Light	20	101	3	60
CO2	8	1100	4	32
UV Index	4	11010	5	20
Wind Speed	6	11011	5	30
Humidity	10	1110	4	40
Rainfall	12	1111	4	48
TOTAL	**125**	**—**	**—**	**325 bits**

Set B total bits = 325.

Q4(c) — Fractional Knapsack [3 marks]

Set A (capacity = 15 units): Rank by impact/frequency (= impact per unit transmitted).

Sensor	Freq	Impact	Density	Action	Impact gained
UV Index	3	3	1.000	Take all 3	3.0
Rainfall	7	6	0.857	Take all 7	6.0
Wind Speed	5	4	0.800	Take all 5	4.0
Pressure	12	8	0.667	Capacity full (0 left)	—

Set A: transmit UV(3), Rainfall(7), Wind(5) = 15 units. Max impact = 3+6+4 = 13.

Set B (capacity = 20 units):

Sensor	Freq	Impact	Density	Action	Impact gained
UV Index	4	3	0.750	Take all 4	3.0
Rainfall	12	7	0.583	Take all 12	7.0
Wind Speed	6	5	0.833	Take all 6 (first)	5.0
CO2	8	6	0.750	Take all 8 → but check order below	—

Corrected Set B ranking: Wind(0.833) > UV(0.750) = CO2(0.750) > Rainfall(0.583) > Hum(0.700)... Recalculate: Hum=7/10=0.700.
Sorted: Wind:0.833, Hum:0.700(?wait Impact=7 for Hum B)... Set B impacts: T:12,H:7,P:9,L:8,CO2:6,W:5,R:7,UV:3.
Densities: T=12/50=0.24, Hum=7/10=0.70, Pres=9/15=0.60, Light=8/20=0.40, CO2=6/8=0.75, Wind=5/6=0.833, Rain=7/12=0.583, UV=3/4=0.75.
Sorted: Wind(0.833), CO2(0.75)=UV(0.75), Hum(0.70), Rain(0.583), Pres(0.60), Light(0.40), Temp(0.24).
Take Wind(6,impact=5)+CO2(8,impact=6)+UV(4,impact=3) = 18 units, impact=14. Remaining 2 units: take 2/10 of Hum, impact=1.4. Total = 20 units, **max impact = 15.4**.

Question 5 — LCS and Edit Distance (Summer 2025)

Read = ACGTA (m=5) for both sets. Set A Ref = AGCTG (n=5). Set B Ref = GACGC (n=5).

Q5(a) — LCS DP Table — Set A [4 + 1 marks]

Recurrence: dp[0][*]=dp[*][0]=0; if Read[i-1]=Ref[j-1] then dp[i][j]=1+dp[i-1][j-1]; else dp[i][j]=max(dp[i-1][j], dp[i][j-1])

		A	G	C	T	G
	0	0	0	0	0	0
A	0	1	1	1	1	1
C	0	1	1	2	2	2
G	0	1	2	2	2	3
T	0	1	2	2	3	3
A	0	1	2	2	3	3

Set A: LCS length = 3. One valid LCS: 'AGT' (A at col1/row1, G at col2/row3, T at col4/row4).

Q5(a) — LCS DP Table — Set B (Ref = GACGC) [4 + 1 marks]

		G	A	C	G	C
	0	0	0	0	0	0
A	0	0	1	1	1	1
C	0	0	1	2	2	2
G	0	1	1	2	3	3
T	0	1	1	2	3	3
A	0	1	2	2	3	3

Set B: LCS length = 3. One valid LCS: 'ACG' (A at col2/row1, C at col3/row2, G at col4/row3).

Q5(b) — Edit Distance DP Table — Set A [3 + 1 marks]

Recurrence: dp[i][0]=i; dp[0][j]=j; if match dp[i][j]=dp[i-1][j-1]; else 1+min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])

		A	G	C	T	G
	0	1	2	3	4	5
A	1	0	1	2	3	4
C	2	1	1	1	2	3
G	3	2	1	2	2	2
T	4	3	2	2	2	3
A	5	4	3	3	3	3

Set A: Edit Distance = dp[5][5] = 3 (3 operations to transform ACGTA → AGCTG).

Q5(b) — Edit Distance DP Table — Set B (Ref = GACGC) [3 + 1 marks]

		G	A	C	G	C
	0	1	2	3	4	5
A	1	1	1	2	3	4
C	2	2	2	1	2	3
G	3	2	3	2	1	2

		G	A	C	G	C
T	4	3	3	3	2	2
A	5	4	3	4	3	3

Set B: Edit Distance = dp[5][5] = 3

Q5(c) — Prove EditDistance ≤ m + n − 2 × LCS [1 mark]

Proof: Let L = LCS(Read, Ref), m = |Read|, n = |Ref|.

Given any LCS of length L, construct the following transformation from Read to Ref:

• **Delete** the (m – L) characters of Read that are not part of the LCS. Cost: m – L.

• **Insert** the (n – L) characters of Ref that are not part of the LCS. Cost: n – L.

Total operations for this specific strategy = (m–L)+(n–L) = m+n–2L.

Since EditDistance is the *minimum* over all possible operation sequences: **EditDistance ≤ m+n–2·LCS**. ■

Verification (Set A): LCS=3, m=5, n=5. RHS = 5+5–6 = 4. EditDistance = 3 ≤ 4. ✓

Verification (Set B): LCS=3, m=5, n=5. RHS = 4. EditDistance = 3 ≤ 4. ✓

FALL 2025 (Set A & B) — Full Marks: 35

Question 1 — DFS, SCC, Topological Analysis (Fall 2025)

Set A graph edges (from figure): A→C, C→A (cycle), A→B, B→D, D→B (cycle), B→E, E→F, E→H, F→G

Set B graph edges: X→Z, Z→X (cycle), X→S, S→Y, Y→T, T→S (cycle?), S→U, U→W — verify with your figure.

Q1(a) — DFS Discovery and Finish Times, start A [3 marks]

Alphabetical tie-break. A(d=1)→B(d=2)→D(d=3): D→B is GRAY → back edge found (cycle!), finish D(f=4). B→E(d=5)→F(d=6)→G(d=7): finish G(f=8), finish F(f=9). E→H(d=10): finish H(f=11), finish E(f=12), finish B(f=13). A→C(d=14): C→A is GRAY → back edge (cycle!), finish C(f=15), finish A(f=16).

Vertex	A	B	C	D	E	F	G	H
Discovery d	1	2	14	3	5	6	7	10
Finish f	16	13	15	4	12	9	8	11
Back edges found	C→A	—	—	D→B	—	—	—	—

Q1(b) — Kosaraju's SCC Algorithm (reuse DFS above) [3 marks]

Descending finish order: A(16), C(15), B(13), E(12), H(11), F(9), G(8), D(4)

Transpose G^T edges: C→A, A→C, B→A (from A→B), D→B (from B→D), B→D (from D→B), E→B, F→E, H→E, G→F

DFS on G^T in descending finish order:

1. Start A: A→C (in G^T, since C→A was original). C unvisited → visit C → C→A(visited). Finish. **SCC■ = {A, C}**
2. Start B: B→D (in G^T, since D→B was original). D unvisited → visit D → finish. **SCC■ = {B, D}**
3. Start E: E→B(visited). **SCC■ = {E}**
4. Start H: H→E(visited). **SCC■ = {H}**
5. Start F: F→E(visited). **SCC■ = {F}**
6. Start G: G→F(visited). **SCC■ = {G}**

SCC	Nodes	Reason
SCC■	{A, C}	Mutual edges A→C and C→A
SCC■	{B, D}	Mutual edges B→D and D→B
SCC■	{E}	No return path
SCC■	{H}	No return path
SCC■	{F}	No return path
SCC■	{G}	No return path

Q1(c) — Minimum Edges to Remove for Topological Order [2 marks]

A topological order exists iff the graph is a DAG. Each cycle must be broken by removing at least one edge.

Cycles found:

• {A, C}: one edge (e.g. C→A or A→C) must be removed.

• {B, D}: one edge (e.g. D→B or B→D) must be removed.

Minimum edges to remove = 2 (one per distinct cycle). Example: remove C→A and D→B.

Q1(d) — Condensation DAG — Any Remaining Cycles? [2 marks]

Super-nodes: {A,C}, {B,D}, {E}, {F}, {G}, {H}

Inter-SCC edges: {A,C}→{B,D} (from A→B); {B,D}→{E} (from B→E); {E}→{F}; {E}→{H}; {F}→{G}

No cycles remain. This is the SCC-condensation theorem: collapsing every SCC into one node always yields a DAG. All inter-SCC edges point forward; no path can return to any previously visited super-node.

Condensation DAG: {A,C} → {B,D} → {E} → {F} → {G}, {E} → {H} (all one-directional, no back-paths).

Question 2 — Highway Network: MST and Toll Coupon (Fall 2025)

Q2(a) — Kruskal's MST [4 marks]

Set A nodes: a, b, c, d, e, f, g, h, i
Edges from figure: a-b:6, a-c:5, b-c:12, b-d:14, c-g:1, c-e:9, d-f:8, e-h:15, e-i:10, f-i:11, g-h:2, d-e:3(?), h-i:... verify exact weights from your paper.

Step	Edge	Weight	Action	Component
1	c–g	1	Add	{ c,g }
2	g–h (or a–c)	2 (or 5)	Add	{ c,g,h } or { a,c,g }
3	a–c	5	Add (if not already)	{ a,c,g,h }
4	a–b	6	Add	{ a,b,c,g,h }
5	d–f or d–e	3–8	Add min connecting	Extends component
...	...	...	Continue until 9 nodes, 8 edges	All connected

Sum the 8 chosen edge weights for the total MST cost. With c–g:1, g–h:2, a–c:5, a–b:6 already selected (sum=14) plus 4 more edges connecting d,e,f,i — read exact weights from your figure.

Set B graph edges: a-b:6, a-c:5, b-c:2, b-d:10, c-e:9, c-g:5, d-e:3, d-f:4, e-h:13, f-i:4(?), g-h:6, h-i:... (verify)

Step	Edge	Weight	Action
1	b–c	2	Add — {b,c}
2	d–e	3	Add — {d,e}
3	d–f or f–i	4	Add — extends {d,e,f}
4	a–c	5	Add — {a,b,c}
5	c–g or a–b	5/6	Add next min
...	...	...	Continue to 8 edges

Q2(b) — Can Multiple MSTs Have the Same Cost? [2 marks]

Answer: Yes.

If a graph has two or more edges of equal weight, multiple valid MSTs may exist with the same total cost. **Simple example:** Triangle X–Y–Z with X-Y:2, Y-Z:2, X-Z:2. Any two of the three edges form an MST with total cost 4. Three distinct MSTs exist, all optimal.

Q2(c) — Toll Coupon: Is Friend's Approach Correct? [4 marks]

Answer: NO — the approach fails in general.

Friend's strategy: Run standard Dijkstra → identify the single highest-toll edge on the shortest path → apply coupon to that edge.

Counterexample proving it fails:

Graph	Edge	Toll
S→A	S to A	10
A→D	A to D	1
S→B	S to B	3
B→D	B to D	5

Standard Dijkstra: shortest path is S→B→D, total cost = 3+5 = 8. Highest toll edge = B→D (5). Friend applies coupon → cost = 3+0 = **3**.

But path S→A→D with coupon on S→A: cost = 0+1 = **1**. This is cheaper, and the friend's method misses it entirely.

Correct approach: Run Dijkstra on an expanded state space — state = (node, coupon_used: bool). This considers all possible (path, coupon-edge) combinations and finds the true minimum.

Question 3 — Huffman Encoding (Fall 2025)

Set A characters: E:4, D:5, U:8, C:10, R:12, B:18, A:30

Set B characters: H:3, K:5, N:7, S:9, L:11, T:16, A:28

Q3(a) — Multiple Huffman Trees from Unique Frequencies? [2 marks]

Answer: Yes — multiple valid Huffman trees are possible even with all-unique frequencies.

Ties can arise in the priority queue between individual nodes and merged subtrees. Different tie-breaking choices (which node goes left/right, which pair to merge first) produce differently-shaped trees. **Bit patterns differ** between trees. **Optimality is unaffected** — every valid Huffman tree achieves the same minimum total weighted path length (total bits). Code lengths per symbol may differ slightly, but the total cost is identical.

Q3(b) — Huffman Simulation — Set A [4 marks]

Step	Merge	Priority Queue after (sorted asc)
Init	—	E:4, D:5, U:8, C:10, R:12, B:18, A:30
1	E(4)+D(5)→ED:9	U:8, ED:9, C:10, R:12, B:18, A:30
2	U(8)+ED(9)→UED:17	C:10, R:12, UED:17, B:18, A:30
3	C(10)+R(12)→CR:22	UED:17, B:18, CR:22, A:30
4	UED(17)+B(18)→UEDB:35	CR:22, A:30, UEDB:35
5	CR(22)+A(30)→CRA:52	UEDB:35, CRA:52
6	UEDB(35)+CRA(52)→ROOT:87	ROOT:87

```
ROOT (87)
0 ■■ UEDB (35)
■ 0 ■■ UED (17)
■ ■ 0 ■■ U (8) → code: 000
■ ■ 1 ■■ ED (9)
■ ■ 0 ■■ E (4) → code: 0010
■ ■ 1 ■■ D (5) → code: 0011
■ 1 ■■ B (18) → code: 01
1 ■■ CRA (52)
0 ■■ CR (22)
■ 0 ■■ C (10) → code: 100
■ 1 ■■ R (12) → code: 101
1 ■■ A (30) → code: 11
```

Char	Freq	Code	Len	Freq × Len
A	30	11	2	60
B	18	01	2	36
R	12	101	3	36
C	10	100	3	30
U	8	000	3	24
D	5	0011	4	20
E	4	0010	4	16
TOTAL	**87**	**—**	**—**	**222 bits**

Set A total bits = 222.

Q3(b) — Huffman Simulation — Set B (H:3, K:5, N:7, S:9, L:11, T:16, A:28)

Step	Merge	Priority Queue after
Init	—	H:3, K:5, N:7, S:9, L:11, T:16, A:28
1	H(3)+K(5)→HK:8	N:7, HK:8, S:9, L:11, T:16, A:28
2	N(7)+HK(8)→NHK:15	S:9, L:11, T:16, NHK:15, A:28
3	S(9)+L(11)→SL:20	T:16, NHK:15, SL:20, A:28
4	NHK(15)+T(16)→NHKT:31	SL:20, A:28, NHKT:31
5	SL(20)+A(28)→SLA:48	NHKT:31, SLA:48
6	NHKT(31)+SLA(48)→ROOT:79	ROOT:79

```
ROOT (79)
0 ■■ NHKT (31)
■ 0 ■■ NHK (15)
■ ■ 0 ■■ N (7) → code: 000
■ ■ 1 ■■ HK (8)
■ ■ 0 ■■ H (3) → code: 0010
■ ■ 1 ■■ K (5) → code: 0011
■ 1 ■■ T (16) → code: 01
1 ■■ SLA (48)
0 ■■ SL (20)
■ 0 ■■ S (9) → code: 100
■ 1 ■■ L (11) → code: 101
1 ■■ A (28) → code: 11
```

Char	Freq	Code	Len	Freq × Len
A	28	11	2	56
T	16	01	2	32
L	11	101	3	33
S	9	100	3	27
N	7	000	3	21
K	5	0011	4	20
H	3	0010	4	12
TOTAL	**79**	**—**	**—**	**201 bits**

Set B total bits = 201.

Q3(c) — Extra Bits: 8-bit vs Huffman [2 marks]

Set A message: 'BRACU'

Char	Huffman Code	Huffman Bits
B	01	2
R	101	3
A	11	2
C	100	3
U	000	3
Total	**—**	**13 bits**

8-bit encoding: 5 × 8 = **40 bits**. Huffman: **13 bits**. **Extra bits = 40 − 13 = 27.**

Set B message: 'THANKS'

Char	Huffman Code	Huffman Bits
T	01	2
H	0010	4
A	11	2
N	000	3
K	0011	4
S	100	3
Total	**—**	**18 bits**

8-bit encoding: 6 × 8 = **48 bits**. Huffman: **18 bits**. **Extra bits = 48 – 18 = 30.**

Q3(d) — Higher-Frequency Always Shorter Codeword? [2 marks]

Answer: Not always true in the strictest sense.

Huffman encoding guarantees no higher-frequency symbol receives a *strictly longer* code than a lower-frequency one. However, two symbols of *different* frequencies can receive codes of the *same* length — for example in Set A, B(18) and A(30) both get 2-bit codes. The algorithm optimises total cost; it does not impose a strict bijection between frequency rank and code length. The statement is **false** as written (equal-length codes for unequal frequencies are allowed).

Question 4 — LCS Table Analysis (Fall 2025 Set A & B)

The given LCS table (for X = X1X2X3X4, Y = Y1Y2Y3Y4):

i\j	–	X1	X2	X3	X4
–	0	0	0	0	0
Y1	0	0	0	0	0
Y2	0	0	0	1	1
Y3	0	0	1	1	2
Y4	0	1	1	1	2

(Set A: rows = Y, cols = X. Set B has the transposed orientation — rows = X, cols = Y — same analysis applies.)

Q4(a) — Direction of Underlined Cells [2 marks]

Cells with underlined values are those where the LCS count *increased* — a character match occurred at that (i,j) position.

Cell	Value	Why it increased	Direction
Y2, X3 → dp[2][3]	1	Y2 = X3 matched → 1+dp[1][2]=1	Diagonal ■ (from dp[1][2])
Y3, X2 → dp[3][2]	1	Y3 = X2 matched → 1+dp[2][1]=1	Diagonal ■ (from dp[2][1])
Y3, X4 → dp[3][4]	2	Y3 = X4? Check: came from dp[2][3]+1=2	Diagonal ■ (from dp[2][3])
Y4, X1 → dp[4][1]	1	Y4 = X1 matched → 1+dp[3][0]=1	Diagonal ■ (from dp[3][0])

Underlined cells always point diagonally up-left (■) — they represent character matches and derive their value from dp[i-1][j-1] + 1.

Q4(b) — Construct String X/Y from Rightmost 4 Digits of Student ID [2 marks]

This is ID-specific. Method demonstrated for rightmost 4 digits = **1905** (from '20251905'):

Set A: String Y = rightmost 4 digits. Set Y = '1905'. From the table LCS=2. Need String X (length 4) such that the given table values hold — meaning Y2=X3 and Y3 somewhere matches X2 or X4.

Construct X so that X[2]=Y[2]='9' and X[4]=Y[4]='5', and other characters don't create additional matches.
Sample: **X = '2905'**. Verify: Y='1905', X='2905'. LCS: '9' and '5' match → LCS=2. Table cell Y2('9'),X3('0') — no match.
Adjust: X='6195' → Y1='1'=X2='1' ✓, Y3='0'=none, Y4='5'=X4='5' ✓ → LCS=2 ✓.

For Set B (String X = rightmost 4 digits, construct Y): same process mirrored.

Always verify your constructed string by filling the full DP table and checking all values match the given table.

Q4(c) — Iterative vs Recursive LCS [1 mark]

Iterative (bottom-up DP) is more efficient in practice.

Both achieve O(m·n) time, but iterative DP avoids function-call overhead, prevents stack overflow for large inputs (recursion stack depth = O(m+n)), and can be space-optimised to O(min(m,n)) by keeping only two rows. Naive recursion without memoisation degrades to $O(2^{m+n})$. Memoised recursion equals iterative in time but retains the stack-depth risk. **Choose iterative DP.**

END OF SOLUTIONS

Summary of key results:

Exam	Q	Key Answer / Result
Spring 2025	Q1c	SCCs: {A},{B},{C},{D},{E},{F},{G,H}
Spring 2025	Q3b	Total bits (Set A) = 153
Spring 2025	Q3c	Max value = 54.46 (knapsack 30s)

Exam	Q	Key Answer / Result
Summer 2025	Q4b	Set A total bits = 292; Set B = 325
Summer 2025	Q4c	Set A max impact = 13 (15 units); Set B ≈ 15.4 (20 units)
Summer 2025	Q5	LCS(ACGTA,AGCTG)=3; EditDist=3; Proof: ED ≤ m+n−2L
Fall 2025	Q1	SCCs: {A,C},{B,D},{E},{F},{G},{H}; remove 2 edges
Fall 2025	Q2c	Friend's coupon approach is WRONG — counterexample given
Fall 2025	Q3b	Set A total bits=222; Set B total bits=201
Fall 2025	Q3c	Set A BRACU: 27 extra bits; Set B THANKS: 30 extra bits

Questions involving <Your Student ID> require substituting your actual ID number. All graph simulations assume tie-breaking in alphabetical/numerical order unless otherwise specified. Verify edge weights against your actual printed exam paper.